Mock 2 E02

Warehouse Inventory Synchronization - Delta MERGE

SUBMISSION TEMPLATE

Difficulty	Marks	Tasks	Screenshot rule
Easy	10	4	Multiple allowed per task

Trainee: _____Soumiya Ayyappan_____ Candidate ID: ____10877547_____ Catalog: _____cat_ltm10877547_____

Objective

Synchronize warehouse stock using a single Delta MERGE with matched updates and unmatched inserts.

Evidence policy

SCREENSHOT POLICY: There is NO one-task/one-screenshot limit. For every assessed task, provide as many readable images as needed to establish ALL listed rubric criteria. Where the rubric tests implementation AND its effect, supply both code/configuration screenshots and result screenshots. If code is long, use consecutive clearly labelled Code A, Code B, Code C images; do not shrink or stitch until unreadable. Label images Q_T01_Code_A, Q_T01_Result_A, etc. Keep table/pipeline names, executed statements and relevant returned rows visible. An output screenshot cannot replace required implementation evidence, and code alone cannot replace required output evidence. Never rerun MERGE, DELETE or other mutations solely to capture evidence: capture prior run/history and current read-only validations.

Environment and object names

Use your existing assigned Unity Catalog and schema mock2. Use the approved existing external location; the instructor supplies your trainee-specific ADLS URI. Create a question-specific EXTERNAL volume at that full child URI. Do not create a new catalog, storage credential or external location. Upload original CSV filenames into input/ under the question volume. Store managed Delta tables and pipeline outputs in schema managed storage, not in the volume.

Setting	Required value
Catalog	Your assigned existing catalog
Schema	mock2
External volume	m2_e02_files
Child ADLS path	mock2/e02/files/ beneath your assigned trainee root
Input folder	/Volumes/<catalog>/mock2/m2_e02_files/input/
Required output objects	m2_e02_inventory, m2_e02_updates

Key terms

- matched: an incoming item_id already exists in the target
- not matched: an incoming item_id must be inserted

Source dataset and column dictionary

Provided file(s): E02_inventory_base.csv, E02_inventory_updates.csv

Column	Required type	Use in operations
item_id	STRING	Matching unique item identifier
item_name	STRING	Product description
stock_qty	INTEGER	Current stock quantity
unit_price	INTEGER	Price per unit

Task requirements and evidence

Task 1. Prepare the target and incoming records | 2 marks

1.1 Upload both exact CSV filenames into the question external volume.

1.2 Create a managed Delta target from the six base records and a separately named three-row change table.

1.3 Confirm source-to-target column types and record counts.

Expected result: Target = 6 rows; change input = 3 rows; both have 4 named fields.

Evaluator evidence - every item below must be legible:

A. Setup UI: question volume, location and both filenames.

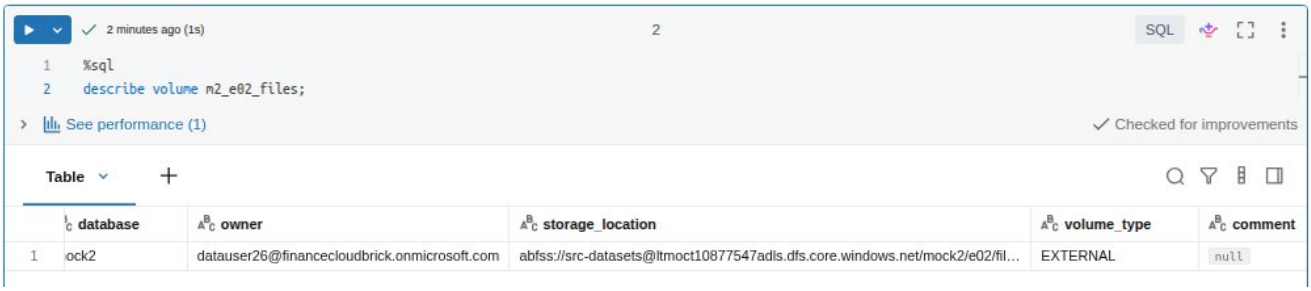
B. Code: typed loads and target/source table creation.

C. Result: named target and incoming table counts 6 and 3.

Use filenames such as M2_E02_T01_Code_A.png, M2_E02_T01_Result_A.png or M2_E02_T01_UI_A.png. Add _B / _C where required. These are naming examples, NOT a fixed screenshot count.

SCREENSHOT PLACEHOLDER - paste the required task evidence below; duplicate this holder for additional screenshots.





[Paste readable screenshot here]

```
    StructField("item_name",StringType(),True),
    StructField("stock_qty",IntegerType(),True),
    StructField("unit_price",IntegerType(),True),
])
df=spark.read.csv("/Volumes/cat_ltm10877547/mock2/m2_e02_files/input/E02_inventory_base.csv",header=True,
schema=schema)
df.write.format("delta").mode("overwrite").saveAsTable("cat_ltm10877547.mock2.m2_e02_inventory")
```

See performance (1) ✓ Checked for improvements

df: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, item_name: string ... 2 more fields]

4 minutes ago (3s) 4

```
from pyspark.sql.functions import *
from pyspark.sql.types import *
schema=StructType([
    StructField("item_id",StringType(),True),
    StructField("item_name",StringType(),True),
    StructField("stock_qty",IntegerType(),True),
    StructField("unit_price",IntegerType(),True),
])
df2=spark.read.csv("/Volumes/cat_ltm10877547/mock2/m2_e02_files/input/E02_inventory_updates.csv",
header=True,schema=schema)
```

```
df2.write.format("delta").mode("overwrite").saveAsTable("cat_ltm10877547.mock2.m2_e02_updates")
```

See performance (1) ✓ Checked for improvements

df2: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, item_name: string ... 2 more fields]

Just now (1s) 5 Python

```
print(df.count())
print("\n",df2.count())
print("\n",len(df.columns))
print("\n",len(df2.columns))
```

See performance (2) ✓ Checked for improvements

```
6

3

4

4
```

Additional screenshots (Code B / Result B / UI B, etc.) may be inserted after this placeholder.

Evidence reference / workspace object: _____

Task 2. Perform inventory synchronization | 3 marks

2.1 Match source and target using item_id.

2.2 For a matched ID replace stock_qty and unit_price with incoming values while retaining the item identity and description.

2.3 For an unmatched ID insert all four incoming columns. Do not delete target rows.

Expected result: I102 stock/price = 18/850; I104 = 9/1250; I107 inserted = 14/950; final 7 rows.

Evaluator evidence - every item below must be legible:

A. Code A/B as needed: complete MERGE match, update and insert branches.

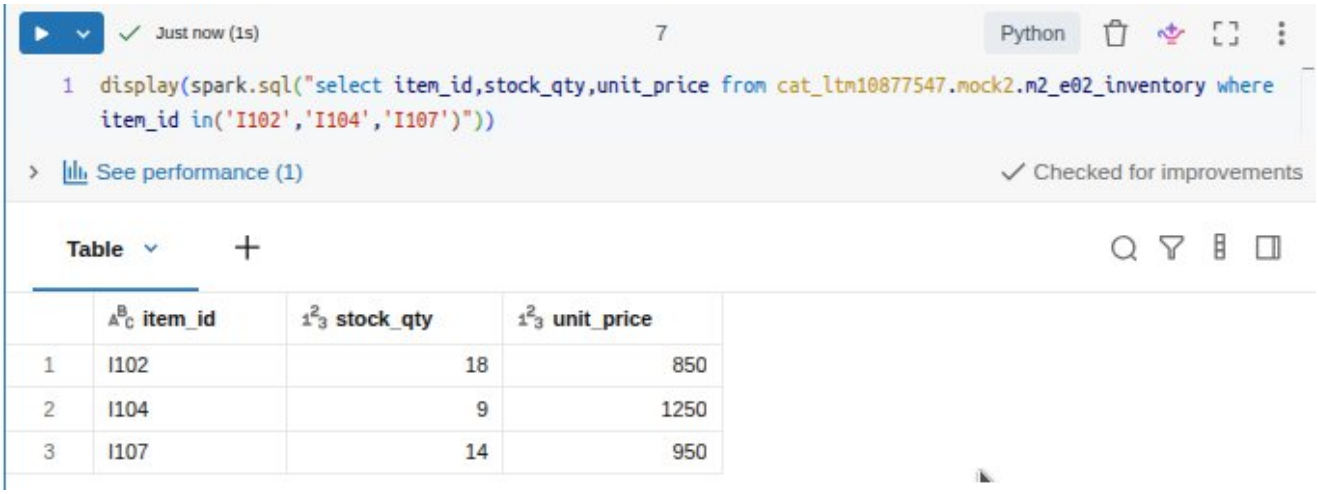
B. Execution evidence: successful mutation once; available MERGE metrics/history.

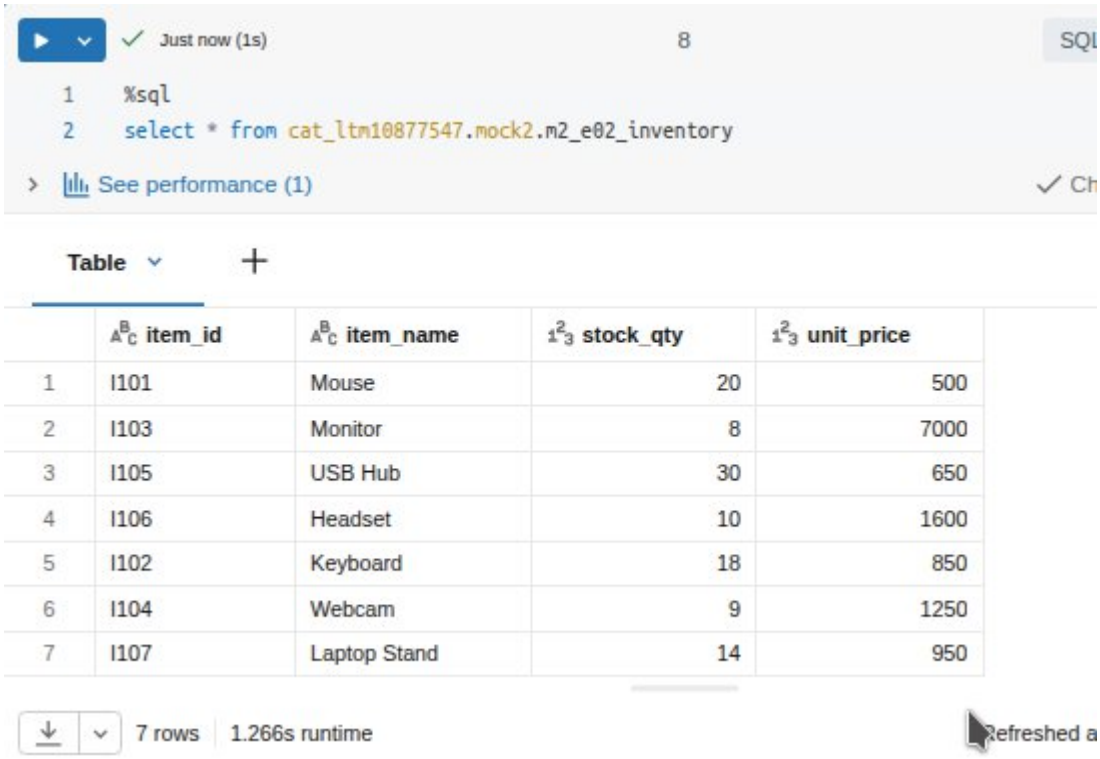
Use filenames such as M2_E02_T02_Code_A.png, M2_E02_T02_Result_A.png or M2_E02_T02_UI_A.png. Add _B / _C where required. These are naming examples, NOT a fixed screenshot count.

SCREENSHOT PLACEHOLDER - paste the required task evidence below; duplicate this holder for additional screenshots.



[Paste readable screenshot here]





	item_id	item_name	stock_qty	unit_price
1	I101	Mouse	20	500
2	I103	Monitor	8	7000
3	I105	USB Hub	30	650
4	I106	Headset	10	1600
5	I102	Keyboard	18	850
6	I104	Webcam	9	1250
7	I107	Laptop Stand	14	950

Additional screenshots (Code B / Result B / UI B, etc.) may be inserted after this placeholder.

Evidence reference / workspace object: _____

Task 3. Validate all changes | 3 marks

3.1 Read the three affected item IDs from the persisted target.

3.2 Show item_name, stock_qty and unit_price, ordered by item_id.

3.3 Confirm exactly two existing rows were updated and one new item inserted.

Expected result: I102 Keyboard 18/850; I104 Webcam 9/1250; I107 Laptop Stand 14/950.

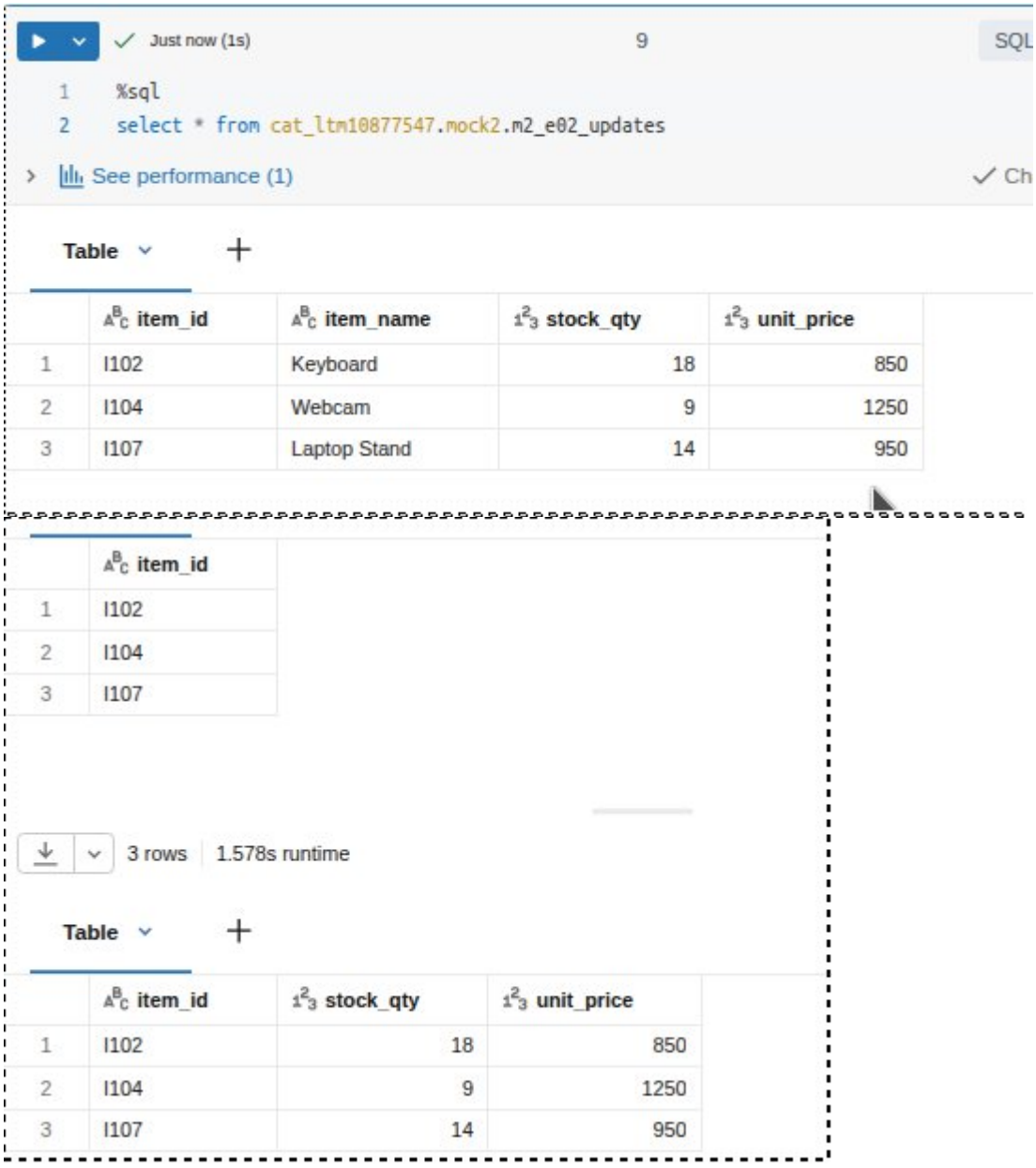
Evaluator evidence - every item below must be legible:

A. Code: validation reading the stored Delta target.

B. Result: all three changed/inserted records and values.

Use filenames such as M2_E02_T03_Code_A.png, M2_E02_T03_Result_A.png or M2_E02_T03_UI_A.png. Add _B / _C where required. These are naming examples, NOT a fixed screenshot count.

SCREENSHOT PLACEHOLDER - paste the required task evidence below; duplicate this holder for additional screenshots.



[Paste readable screenshot here]

Additional screenshots (Code B / Result B / UI B, etc.) may be inserted after this placeholder.

Evidence reference / workspace object: _____

Task 4. Reconcile final inventory | 2 marks

4.1 Confirm target row count and preservation of an unaffected record.

4.2 Show I101 retains stock_qty 20 and unit_price 500; verify no preexisting item was deleted.

Expected result: Final 7 rows; I101 Mouse 20/500; six original IDs remain plus I107.

Evaluator evidence - every item below must be legible:

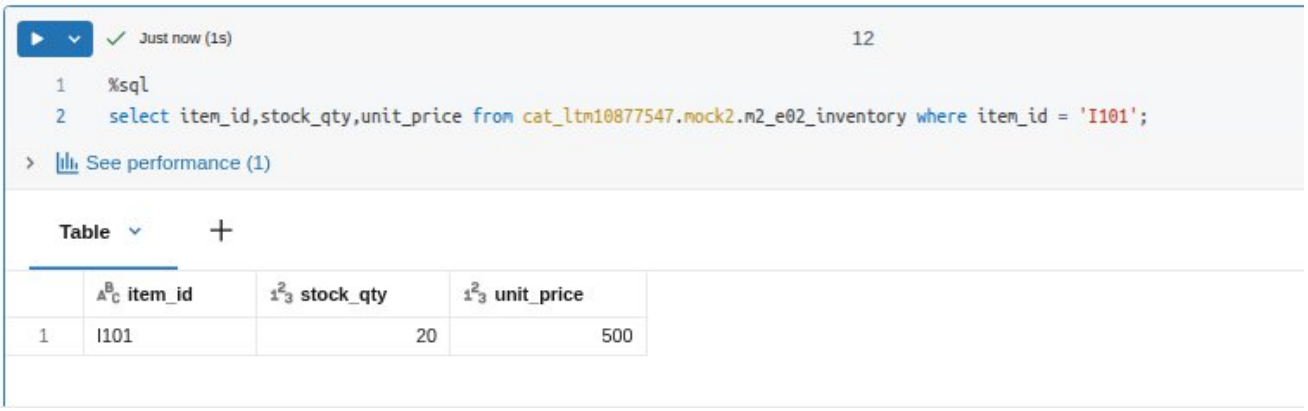
A. Code: final row-count and unchanged-record checks.

B. Result: complete count and I101 values.

Use filenames such as M2_E02_T04_Code_A.png, M2_E02_T04_Result_A.png or M2_E02_T04_UI_A.png. Add _B / _C where required. These are naming examples, NOT a fixed screenshot count.

SCREENSHOT PLACEHOLDER - paste the required task evidence below; duplicate this holder for additional screenshots.





[Paste readable screenshot here]

Additional screenshots (Code B / Result B / UI B, etc.) may be inserted after this placeholder.

Evidence reference / workspace object: _____

Expected final results

Task 1: Target = 6 rows; change input = 3 rows; both have 4 named fields.

Task 2: I102 stock/price = 18/850; I104 = 9/1250; I107 inserted = 14/950; final 7 rows.

Task 3: I102 Keyboard 18/850; I104 Webcam 9/1250; I107 Laptop Stand 14/950.

Task 4: Final 7 rows; I101 Mouse 20/500; six original IDs remain plus I107.

Evaluation rubric and evidence map

Task	Rubric metric - evidenced by screenshots	Marks
1	Setup UI: question volume, location and both filenames.	1
1	Code: typed loads and target/source table creation.	1
1	Result: named target and incoming table counts 6 and 3.	0
2	Code A/B as needed: complete MERGE match, update and insert branches.	2
2	Execution evidence: successful mutation once; available MERGE metrics/history.	1
3	Code: validation reading the stored Delta target.	2
3	Result: all three changed/inserted records and values.	1
4	Code: final row-count and unchanged-record checks.	1
4	Result: complete count and I101 values.	1
TOTAL	No hidden checks or extra requirements	10

Submission requirements

Submit this matching template populated with readable taskwise screenshots. Provide 2 or 3 screenshots within a task when the implementation is long, or when code/configuration and results cannot both fit legibly. Show every assessed rubric metric, not just a success message. No fixed screenshot count, no scripts supplied in assessment instructions. Keep the executed workspace artifacts and history available. Do not rerun data-changing operations to recapture evidence.